



SORBONNE UNIVERSITÉ
M1 INFORMATIQUE - STL

OUVERTURE

Algorithmes de Rémy et Arbres ABR

Devoir de programmation

Benatsi Feriel
Mebarkia Maya
Zeghmiche Dalila

23 décembre 2025

Table des matières

1	Introduction	2
2	Analyse algorithmique et implémentation	2
2.1	Prototype de l'algorithme de Rémy	2
2.2	Algorithme de Rémy optimisé	2
2.3	Algorithme de génération d'un arbre ABR	3
3	Etude expérimentale	4
3.1	Analyse de la hauteur	4
3.2	Analyse de la largeur	5
3.3	Analyse de la profondeur moyenne des feuilles d'un arbre	6
3.4	Analyse de l'équilibre par rapport à la racine	7
3.5	Validation de la complexité attendue	8
4	Conclusion	9

1 Introduction

Ce projet s'inscrit dans le cadre de l'UE Ouverture et vise à se familiariser avec l'algorithmique en utilisant le langage OCaml. L'objectif principal est d'implémenter et comparer deux algorithmes distincts permettant de générer des arbres binaires de taille n : l'algorithme de Rémy et la génération par insertion aux feuilles (type ABR). Au-delà de la simple implémentation, une contrainte forte de complexité linéaire est imposée, nécessitant une réflexion plus approfondie sur les structures de données utilisées. Enfin, une étude expérimentale rigoureuse permettra de mettre en évidence les différences structurelles entre les deux modèles et de vérifier si les deux approches sont équivalentes.

2 Analyse algorithmique et implémentation

Pour commencer, nous aimerions rappeler que ce projet repose sur l'implémentation des algorithmes dans le paradigme fonctionnel. Dans le fichier `arbre_binaire.ml`, nous avons défini la structure de base par le type somme récursif : `type arbre_binaire = Feuille | Noeud of arbre_binaire * arbre_binaire`. Cette définition simple nous a permis d'écrire naturellement les fonctions de mesure comme `hauteur` ou `largeur`.

Cependant, une approche impérative a été suivie pour atteindre la complexité linéaire souhaitée dans le cas des algorithmes de Rémy (optimal) et de génération ABR. L'utilisation de tableaux et de références a été nécessaire pour éviter les copies récursives coûteuses. Cette démarche hybride nous permet ainsi de concilier la clarté du modèle fonctionnel avec l'efficacité des accès mémoire en temps constant.

2.1 Prototype de l'algorithme de Rémy

Le prototype de Rémy suit la logique suivante : pour passer d'un arbre de taille k à $k+1$, on choisit uniformément un noeud existant et on le remplace. Notre implémentation dans `remy_prototype.ml` s'appuie sur la fonction `etape_remy`. Celle-ci calcule d'abord la taille totale de l'arbre, tire une position aléatoire, puis transmet le travail à deux fonctions récursives : `sous_arbre_a_position` et `remplacer_a_position`.

Ces fonctions effectuent un parcours de l'arbre. Pour savoir si la position cherchée est dans le sous-arbre gauche ou droit, nous calculons à chaque noeud la taille de son fils gauche (`total_gauche`). Si l'indice cherché est inférieur à ce total, on descend à gauche ; sinon, on descend à droite en ajustant l'indice. Bien que correcte, cette méthode est lourde : en effet, elle doit recalculer les tailles à chaque étape, ce qui confirme la complexité quadratique $O(n^2)$ observée.

2.2 Algorithme de Rémy optimisé

Pour passer d'une complexité quadratique à une complexité linéaire, l'algorithme ne doit plus parcourir l'arbre en entier à chaque fois. Dans `remy_optimal.ml`, nous avons défini un

type enregistrement `cellule` contenant quatre champs mutables : `gauche`, `droite`, `parent` (des entiers représentant les indices dans le tableau) et un booléen `est_gauche`.

Ce champ `est_gauche` est un détail d'implémentation crucial : il nous permet de mettre à jour le lien du parent vers l'enfant en $O(1)$ sans avoir à vérifier si l'enfant actuel est le fils gauche ou droit. L'optimisation repose sur deux mécanismes clés :

- **Accès direct et mutabilité** : Nous utilisons un tableau `tab` de taille $2n+1$ initialisé avec `Array.init`. Au lieu de reconstruire le chemin vers la racine, nous modifions directement les champs de la `cellule` via son indice.
- **Gestion des candidats en $O(1)$** : Pour choisir un noeud uniformément, nous maintenons un tableau auxiliaire `noeuds`. Lors de l'étape k , nous tirons un indice aléatoire dans ce tableau. L'ajout des deux nouveaux noeuds se fait simplement en les écrivant à la fin du tableau `noeuds` et en incrémentant un compteur `nb_noeuds`.

À la fin de la génération, une fonction `tableau_vers_arbre` reconstruit la structure récursive `arbre_binaire` à partir du tableau pour permettre l'utilisation de nos fonctions d'analyse.

2.3 Algorithme de génération d'un arbre ABR

L'algorithme de génération d'un ABR aléatoire repose sur un principe différent : nous construisons l'arbre par insertions successives. Pour garantir une complexité linéaire, nous ne pouvons pas utiliser l'insertion classique dans un ABR qui coûterait $O(n \log n)$ en moyenne. Nous utilisons alors une approche semblable : on part d'une feuille initiale, et on répète n fois l'opération de substitution d'une feuille choisie uniformément par un noeud interne (portant deux nouvelles feuilles).

Notre implémentation dans `abr.ml` utilise trois tableaux : `gauche` et `droite` pour stocker la structure de l'arbre (indices des fils), et un tableau `feuilles` pour lister les candidats disponibles. L'astuce principale pour maintenir la complexité $O(1)$ se trouve dans la gestion de ce tableau `feuilles`. Lorsqu'une feuille à l'indice k est choisie pour devenir un noeud interne :

1. Nous lisons son identifiant `feuille = feuilles.(k)`.
2. Nous mettons à jour `gauche.(feuille)` et `droite.(feuille)` avec deux nouveaux indices générés par un compteur `prochain`.
3. Pour retirer la feuille choisie du tableau sans décaler tous les éléments (ce qui serait en $O(n)$), nous effectuons un échange avec la dernière feuille active du tableau ("swap with last") : `feuilles.(k) <- feuilles.(nb_feuilles - 1)`.
4. Nous ajoutons ensuite les deux nouvelles feuilles à la fin du tableau.

Cette méthode nous permet de construire un ABR aléatoire de taille n en exactement n itérations, chacune coûtant strictement $O(1)$, assurant ainsi une complexité totalement linéaire. Une fonction finale `construire` convertit cette structure tabulaire en notre type `arbre_binaire`.

3 Etude expérimentale

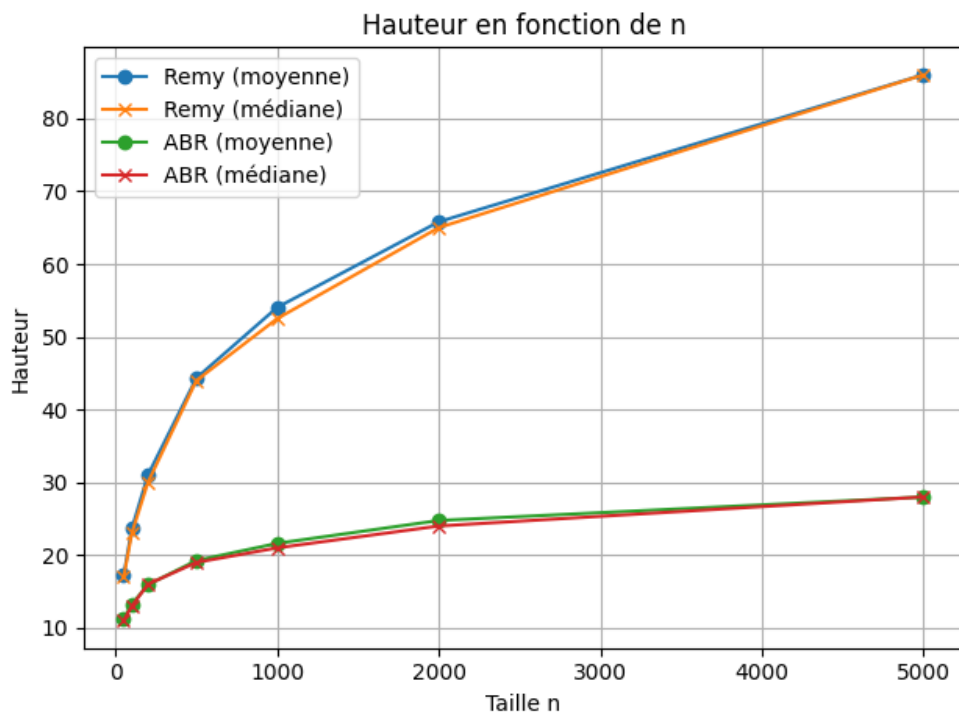
Pour mettre en place les expérimentations et pouvoir comparer les deux modèles aléatoires d'arbres implémentés, il faut d'abord définir un cadre de mesure. Nous avons choisi de faire varier la taille n des arbres sur la plage de valeurs suivantes : 50, 100, 200, 500, 1000, 2000 et jusqu'à 5000 noeuds. Une échelle logarithmique de la sorte nous permet d'observer le comportement asymptotique des algorithmes. Nous effectuerons 50 répétitions indépendantes pour chaque taille n afin d'obtenir des statistiques fiables, moyenne et médiane.

En somme, nous récupérerons cinq mesures caractéristiques pour chaque arbre généré :

- le temps de génération (en ms) qui permet de vérifier la complexité temporelle ;
- la hauteur qui indique l'étirement vertical de l'arbre ;
- la largeur qui indique l'étirement horizontal de l'arbre ;
- la profondeur moyenne des feuilles qui mesure l'équilibre de l'arbre ;
- la taille du sous-arbre gauche qui analyse la symétrie de la racine.

Ces données sont collectées automatiquement dans un fichier CSV `results.csv` qui sera ensuite utilisé pour générer des graphiques utiles pour nos analyses.

3.1 Analyse de la hauteur



En observant les courbes obtenues (Figure 1), nous remarquons une nette différence entre les deux algorithmes, et ce dès les plus petites valeurs de n . Pour l'algorithme de Rémy, la croissance est rapide et linéaire en apparence sur le graphique (mais mathématiquement en racine carrée). Pour $n = 100$, la hauteur dépasse déjà 20. Ce comportement s'accroît avec

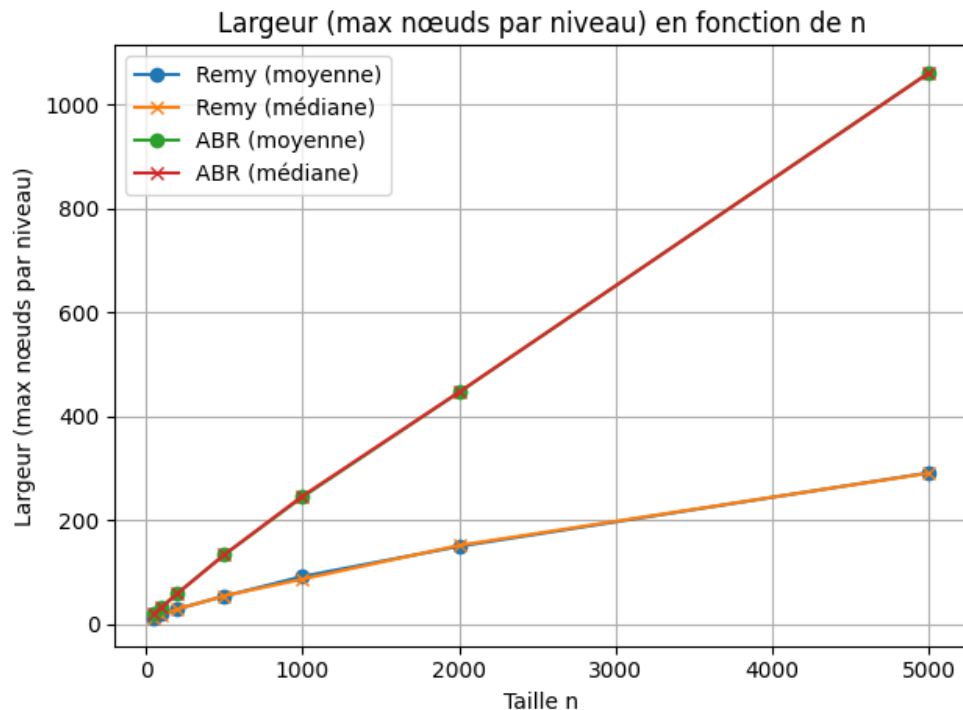
n : les courbes sont d'ailleurs assez "hachurées", témoignant d'une forte variance : d'une exécution à l'autre, la hauteur d'un arbre de Rémy peut varier considérablement. En revanche, la croissance est beaucoup plus lente et régulière (courbe logarithmique) pour l'algorithme de génération ABR. Alors que la taille de l'arbre est multipliée par 50 (passant de 100 à 5000 noeuds), la hauteur ne fait que doubler, atteignant une moyenne d'environ 27 pour $n = 5000$. Cela montre une très grande stabilité structurelle.

L'analyse des ordres de grandeur confirme deux classes de complexité bien distinctes pour la hauteur moyenne $H(n)$:

- Pour l'algorithme de Rémy ($O(\sqrt{n})$) : Si nous prenons $n = 5000$, la théorie prédit un ordre de grandeur proportionnel à $\sqrt{5000} \approx 70.7$. Nos résultats expérimentaux (moyenne environ 85) sont très proches de cet ordre de grandeur, confirmant la nature uniformément distribuée des arbres.
- Pour l'algorithme ABR ($O(\log n)$) : La théorie des ABR aléatoires prédit une hauteur asymptotique proche de $3 \ln(n)$. Pour $n = 5000$, $3 \ln(5000) \approx 25.5$. Nos résultats expérimentaux (autour de 27) collent parfaitement à cette prédiction théorique.

Nous pouvons en conclure que l'algorithme ABR produit des arbres compacts et équilibrés (proches de l'optimal), tandis que l'algorithme de Rémy génère des arbres filiformes et profonds. Cette différence est cruciale : un logiciel testé uniquement sur des ABR pourrait ne pas fonctionner s'il rencontre un arbre de Rémy, beaucoup plus profond.

3.2 Analyse de la largeur

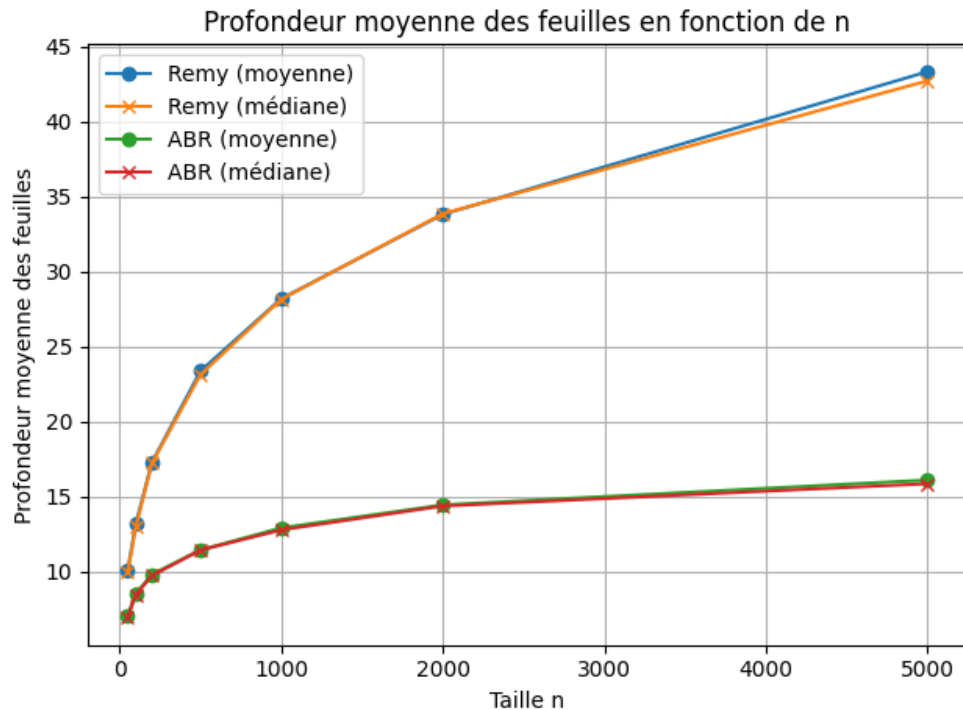


L'étude de la largeur (nombre maximal de noeuds situés à la même profondeur) vient compléter celle de la hauteur. Si la hauteur nous renseigne sur l'étirement vertical de l'arbre, alors la largeur nous indique sa densité horizontale. En examinant les courbes obtenues (Figure 2), nous remarquons une claire inversion des tendances par rapport à l'analyse précédente : ici, l'algorithme ABR domine.

Les résultats expérimentaux montrent que la largeur des arbres de type ABR croît de manière linéaire. Pour $n = 5000$, la largeur moyenne dépasse les 1060 noeuds : cela veut dire qu'à un certain niveau de profondeur, on retrouve plus de 20% des noeuds de l'arbre. Inversement, les arbres générés par le modèle de Rémy restent beaucoup plus "fins" : pour la même taille $n = 5000$, leur largeur moyenne stagne aux alentours des 290 noeuds.

Ces observations concordent avec celles portées précédemment : puisque l'ABR a une hauteur logarithmique soit très faible, il est contraint de "tasser" ses noeuds sur peu de niveaux, ce qui augmente mécaniquement sa largeur.

3.3 Analyse de la profondeur moyenne des feuilles d'un arbre



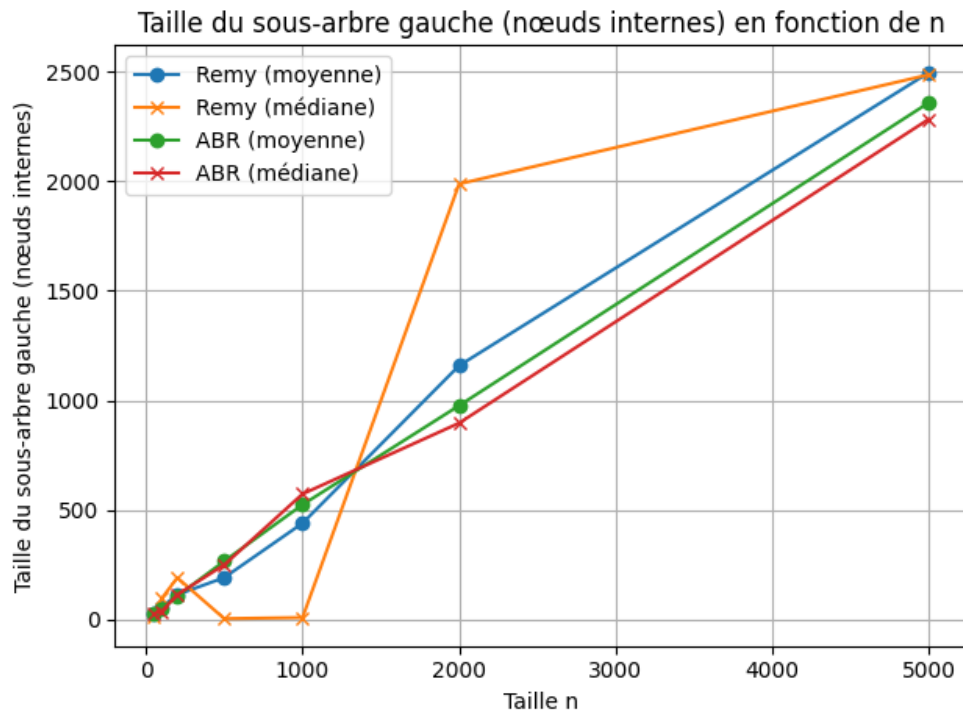
La profondeur moyenne des feuilles est un indicateur global de l'équilibre de l'arbre. Elle représente la longueur du chemin parcouru pour atteindre une feuille choisie au hasard.

L'examen des courbes (Figure 3) révèle une forte divergence entre les deux algorithmes. La profondeur moyenne augmente très lentement pour la génération ABR, passant d'environ

7 pour $n = 50$ à seulement 16 pour $n = 5000$. Cette progression logarithmique montre que la masse des feuilles reste concentrée relativement près de la racine. Nos valeurs obtenues sont très proches des mesures théoriques ($\log_2(5000) \approx 17$), validant la conformité de nos implémentations. A l’opposé, les arbres générés par l’algorithme de Rémy ont une profondeur moyenne beaucoup plus élevée, atteignant environ 43 pour $n = 5000$. Les courbes représentant ses résultats suivent une allure similaire à celles de la hauteur, caractérisées par une croissance en $\sqrt{n}$.

Nous pouvons en retenir que l’arbre ABR optimise naturellement l’accès moyen aux données tandis que l’arbre de Rémy produit des structures plus profondes, donc moins efficaces pour la recherche.

3.4 Analyse de l’équilibre par rapport à la racine



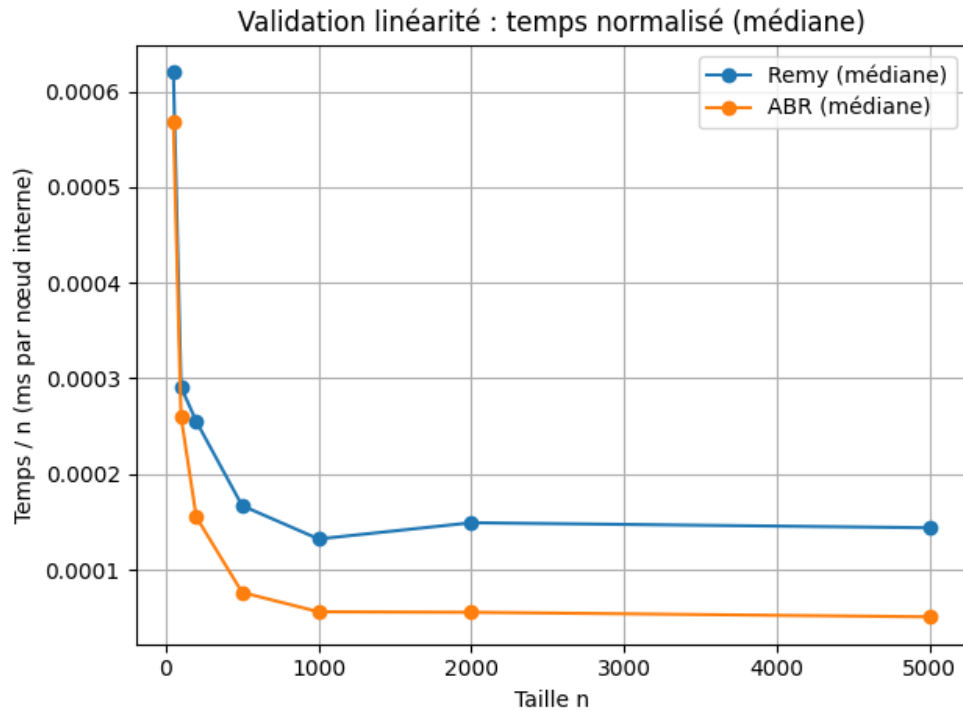
L’étude du sous-arbre gauche d’un arbre nous permet de mesurer sa symétrie car cela revient à regarder comment la racine sépare le total de noeuds entre la gauche et la droite.

Nous pouvons apercevoir une différence fondamentale de comportement. Pour les arbres ABR, la taille du sous-arbre gauche croît linéairement et se stabilise autour de la diagonale $y = x/2$. Les courbes de moyenne et médiane sont confondues, indiquant que la racine joue statistiquement bien son rôle de pivot en séparant l’arbre en deux parties à peu près égales.

Le cas de l’algorithme de Rémy est beaucoup plus singulier. Si la moyenne semble indiquer un équilibre parfait à $n/2$, la médiane révèle une réalité toute autre : elle oscille entre 0 et

n. Cela traduit une distribution bimodale : la racine d'un arbre uniforme a une très forte probabilité d'être extrêmement déséquilibrée, possédant soit un sous-arbre gauche quasi-vide, soit un sous-arbre gauche contenant la quasi-totalité des noeuds.

3.5 Validation de la complexité attendue



Pour valider la linéarité de nos algorithmes, nous avons tracé le temps d'exécution divisé par la taille n (Figure 5). Si un algorithme est linéaire ($T(n) \approx c \cdot n$), alors le ratio $T(n)/n$ doit tendre vers une constante c .

Les courbes obtenues confirment cette hypothèse. Après une phase initiale de stabilisation pour les petites valeurs de n (due aux coûts fixes d'initialisation et à la gestion mémoire d'OCaml), le temps par noeud devient constant pour les deux algorithmes. Pour $n = 5000$, nous observons un temps unitaire stable d'environ 0.00014 ms/noeud pour Rémy et 0.00005 ms/noeud pour l'ABR. Cette stabilité horizontale des courbes valide que nos implémentations sont bien en $O(n)$, et donc respectent la contrainte principale du projet.

4 Conclusion

Ce projet nous a permis d'explorer deux méthodes de génération aléatoire d'arbres binaires et de mettre en évidence leurs différences fondamentales. L'algorithme de Rémy, bien que plus complexe à implémenter en temps linéaire, génère des arbres selon une distribution uniforme sur l'ensemble des arbres possibles. Nos expérimentations ont montré que ces arbres sont structurellement déséquilibrés et hauts.

Au contraire, la génération de type ABR produit des arbres beaucoup plus compacts et larges, avec une hauteur logarithmique. Ces deux modèles ne sont donc pas du tout équivalents : choisir l'un ou l'autre pour tester une application ou simuler des données aura un impact majeur sur les performances observées. Au-delà de l'aspect théorique, ce travail a montré l'importance de l'optimisation algorithmique (passage de $O(n^2)$ à $O(n)$) et la nécessité d'une analyse statistique fine pour interpréter correctement des données expérimentales.